D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Empowered Autonomous Institute)

KASABA BAWADA, KOLHAPUR

D Y PATIL
COLLEGE of
ENGINEERING & TECHNOLOGY
(AN EMPOWERED AUTONOMOUS INSTITUTE)
KASABA BAWADA, KOLHAPUR

A PROJECT SYNOPSIS

ON

OPS-AGENT:
Multi-Agent IT Incident Resolution with Deterministic Governance

Submitted By

Name	Roll No
Vinit Chougale	07-B
Aditya Pandere	13-B
Aditya Pandav	62-B
Satej Patil	63-B
Harshvardhan Itkarkar	75-A

Under the Guidance of

Mr. J. B. Metkari

Assistant Professor

Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)

D Y Patil College of Engineering & Technology

Kasaba Bawada, Kolhapur

2026-27

CONTENTS:

Sr.No.	Particulars	Page Number
1	Introduction	3
2	Literature Survey	6
3	Problem Statement	9
4	Objectives	10
5	Methodology/Planning of work	11
6	Hardware/Software Requirement	15
7	Project Schedule (Timeline)	16
8	References	17

Remarks by Project Guide :(Accepted/Rejected/Suggested for Corrections)

_____

_____

Remarks by Project Panel: (Accepted/Rejected/Suggested for Corrections)

_____

_____

Project Guide **Project Coordinator** **HOD CSE(AIML)**

1. Introduction

In the current digital age, organisations are relying more and more on their IT systems to run their businesses, provide customer service, maintain cloud-based software, cybersecurity, and enterprise software. As enterprises evolve towards cloud-based infrastructures, distributed systems, and DevOps, the number of incidents, service requests, and technical support tickets rises exponentially. IT support teams and Site Reliability Engineering (SRE) teams respond to incidents and technical support tickets to resolve them and provide customer support. The sheer volume of tickets makes responding to every service request a resource-heavy, time-consuming, and costly affair.

Traditional IT service management practices are highly dependent on humans to triage tickets, identify root causes, find relevant documentation, determine the right resolution, and decide if and when an issue requires escalation. As a consequence, incidents can often take a long time to resolve, leading to dissatisfied users, misprioritized workloads, wasted resources, and loss of productivity. Support engineers may have to perform repetitive diagnostic tests, look for documentation, and coordinate with other teams before being able to actually resolve an incident. Furthermore, as organizations grow more complex, these factors become even more challenging and problematic, and create a strong motivation for IT departments to adopt smarter, automated solutions that can help or fully replace humans in many aspects of incident resolution.

Recent developments in Artificial Intelligence (AI) field, especially Large Language Models (LLMs) enabled modern intelligent systems to acquire abilities comparable to humans in a variety of domains. The latest language models can understand natural language, summarize documents, generate natural language responses, retrieve relevant information from knowledge bases, and perform complex multi step reasoning over information. The capabilities of modern AI solutions allow them to address diverse tasks related to enterprise IT support. According to the industry research reports published in 2025, AI enabled customer support solutions decreased the first response time by an average of over 50%. In addition, AI virtual agents could resolve a significant percentage of all customer requests without human assistance. Thus, AI powered solutions have been proven effective in enterprise IT support automation to address various issues.

Despite the benefits, it is important to note that modern support systems utilizing AI have a number of shortcomings. First and foremost, it should be noted that the vast majority of existing solutions solve only simple, well-structured, and routinized problems. The cases which involve multiple systems, lack relevant information, contain contradicting data, require analysis of security related issues or have a strong impact on the business are best handled by humans. Single-agent systems, therefore, may provide irrelevant, inadequate, or even hallucinated responses and thus eliminate the primary purpose of such solutions.

In order to address these limitations, recently, researchers and top tech companies have been turning their attention to Multi-agent Artificial Intelligence. Unlike the first approach, which employs one single agent to fulfill all tasks, multi-agent breaks down the whole work into several specialized parts. In other words, each agent within the system is responsible for a specific area, such as natural language comprehension, knowledge retrieval, solution generation, verification, and governance. The collaboration of these agents enables an improved accuracy, better explainability, scalability, and governance of the overall process.

Several of the major players in the industry such as Microsoft, PagerDuty, ServiceNow, Google, and OpenAI are already taking part in agent based enterprise automation. Recent research from the years of 2025 and 2026 have demonstrated the value of autonomous agents that are able to plan, reason, retrieve information, and coordinate workflows within the larger enterprise. The major caveat to the entire system is that while most of these autonomous agents can generate a suitable response and automate a substantial amount of work for itself, it still has to have reliable decision making processes that are explainable, auditable, privacy preserving, and safe for real world enterprise usage. The companies involved all realize the necessity of creating AI systems that not only generate answers but are confident in said answers, capable of performing a proper risk assessment, proper escalation procedures, as well as proper compliance and governance in adherence to all enterprise level standards.

The proposed project aims to solve the identified business problem by designing an innovative multi-agent incident management system for enterprise IT support. The suggested system would be based on the four agents that would address diverse tasks in the process of addressing incidents and problems. The Intake agent would be responsible for receiving tickets and analyzing the description provided by the user to identify relevant details and issue category. This agent would also determine the priority of the reported incident and prepare the information for the following stage of processing. As a result, there would be less need for manual intervention in the initial stages of processing tickets and categorization.

Once the situation has been clarified, the Retrieval Agent scans through the organization's data, documentation, records, and manuals to find relevant information pertinent to the case. The agent accesses the best information needed to address the situation rather than being dependent solely on the language model’s knowledge. Instead, the agent employs an advanced method known as Retrieval Augmented Generation (RAG), in which relevant documents serve as the basis for the information provided. This approach ensures that the information provided is reliable and reduces hallucination, which would have been the case when using only the Language Model’s information.

The information retrieved is then processed by the Resolution Agent. Resolution Agent uses Large Language Models to reason over the retrieved context and generate structured troubleshooting recommendations, diagnostics, configurations, or resolutions. It combines reasoning with organizational documentation to provide accurate and context-specific answers that are easy to understand for the engineers. Rather than general advice, the Resolution Agent constructs solution plans based on the organization’s information.

A critical innovation in the proposed system is the introduction of the Governance Agent. This agent fundamentally differs from the existing AI support systems in that it continuously assesses the relevance, confidence level, completeness, and safety of the solution before presenting it to the user. It checks if there is enough evidence in favor of the solution, if the solution follows the organization’s rules, if human intervention is needed in the task, and stores all decisions for auditing purposes. These additions will ensure that users are confident in the provided solutions, increasing the trust and reducing implementation risks in using AI in the enterprise..

The project is implemented using modern AI technologies such as Large Language Models, Retrieval Augmented Generation, Vector Databases for Semantic Document Search, Natural Language Processing, Embeddings, and Multi-Agent orchestration framework. For knowledge retrieval, the solution uses vector similarity search that finds relevant information by comparing the meaning of search terms. The system can be easily integrated with enterprise ticketing

systems in the future, such as ServiceNow, Jira Service Management, Zendesk, Freshservice, or Microsoft Dynamics.

The proposed system represents a multidisciplinary concept that integrates artificial intelligence, natural language processing, enterprise IT service management, multi-agent systems, and site reliability engineering domains. The solution encompasses diverse ideas and technological concepts, including machine learning, information retrieval, enterprise software, cloud technologies, and intelligent agents, to operationalize and solve real-life problems related to supporting modern IT organizations' operational needs. Among the broad range of technical concepts and ideas, the primary attention within the scope of the solution is devoted to large language models, retrieval augmented generation based on embedding similarity, semantic search, prompt engineering, autonomous agents, confidence scoring, human-machine coordination, governance, and explainability.

The main point of this project is not simply automation of ticket responses, but rather creating governable, privacy conscious AI, that can scale up to handle real-life incidents that require actual management on the enterprise level. It can be achieved through dividing the work between multiple agents and introducing a governance layer, thus making the system more effective at handling incidents, reducing the response time, less error prone, reducing the workload of the human operators, and promoting the responsible use of AI. Additionally, such design can later be expanded to include additional functionality, such as full resolution of the underlying incidents, prediction of incidents, self healing mechanisms, and more.

Overall, this project is crucial to the development of the next generation intelligent enterprise support systems. The paper presents a comprehensive approach to building systems that utilize Multi-agent collaboration, retrieval based knowledge grounding, and governance-aware decision making to produce AI assisted support solutions that surpass the current generation of such systems. Thus, the research described in this article makes a significant contribution to the field of autonomous enterprise AI.

2. Literature Survey

As multi-agent large language model systems begin to transition from prototypes to production-ready incident response tools, the present work has engaged the question from the perspectives of triage automation, commercialization, governance design, and confidence calibration, over the course of the last two years. This following section recaps the related works that influenced the present proposal.

2.1 Multi-Agent Incident Triage Systems

Microsoft Research proposed Triangle, an end-to-end multi-LLM-agent system for incident triage in large-scale cloud services, which leverages a semantic-distillation mechanism to handle the multi-faceted nature of real-world incidents and demonstrates that a multi-agent pipeline can be utilized reliably for production cloud-scale applications [1]. The key limitation of this study is that the authors only evaluated the performance of their approach in terms of triage accuracy, thus neglecting the governance or escalatory behaviors enacted by the system after a recommendation has been produced.

2.2 Multi-Agent versus Single-Agent Orchestration

A large-scale controlled experiment comprising of 348 trials compared the efficacy of using a single agent and multiple agents for incident response based on a new decision quality metric and found that orchestrated use of multiple agents resulted in significantly more actionable recommendations than a single agent system [2]. Yet, the study found no analysis of the confidence scores reported by either class of models nor any attempt at taxonomy development to classify the failures that were observed.

2.3 Hierarchical Multi-Agent Root-Cause Analysis Frameworks

A hierarchical five-agent Site Reliability Engineering framework coordinates dedicated agents for logs, metrics, traces, and infrastructure via the Model Context Protocol to perform automated root-cause analysis, reporting that traditional threshold-based alerting achieves only 45 to 65 percent root-cause accuracy [3]. This highlights the benefits of reasoning-based alerts over static rule-based alerts, but the framework lacks a formal governance agent, as well as an explicit escalation mechanism for reasoning-based alerts that have low confidence scores

2.4 Security and Robustness of Agentic Systems

A survey on the security duality of LLM agents explores both the avenues by which agentic systems expose new risks and how agents could be leveraged to improve cybersecurity defences [4]. The work serves as a reminder that unchecked autonomy can be a source of insecurity, motivating the development of systems that govern agents rather than being governed by them.

2.5 Commercial Agentic ITSM Platforms

Commercially, PagerDuty's 2026 Operations Cloud introduced an SRE Agent capable of autonomous incident operations with cross-cloud interoperability (e.g., other cloud providers agents) [5], while ServiceNow's acquisition of Moveworks resulted in an EmployeeWorks product family in 2026, which bundled together conversational AI, retrieval-based answers, and agentic ITSM, being shipped as a single product [6]. These platforms confirm that the four-agent pipeline architecture proposed in this project is already commercially available, meaning the architecture alone cannot be the primary contribution of this project; the evaluation rigor applied to its governance layer must be.

2.6 Human-in-the-Loop Governance Standards and Tooling

Open-source tools such as the HumanLayer SDK now provide infrastructure for approval gates and audit trails in agentic systems [7], and industry guidance increasingly formalizes human oversight requirements for autonomous AI deployments [8], [9], [10]. However, this guidance uniformly recommends using the model's own, raw confidence as the criterion for escalation, without addressing whether this signal can be trusted.

2.7 LLM Confidence Calibration and Conformal Prediction

Classical work established that modern neural networks are poorly calibrated [19], and recent results demonstrate that this issue persists in large language models, where verbalized confidence is an unreliable indicator of internal certainty [11]. Conformal prediction, a distribution-free uncertainty-quantification framework with finite-sample coverage guarantees, has since been proposed as a more reliable alternative and has seen recent application in scalable oversight [16], cost-effective agent orchestration [17], and agent error attribution [18], all of which build on foundational results from the conformal prediction literature [20]. Overall, the recent line of work demonstrates the feasibility of achieving calibrated confidence across a variety of domains but has yet to be applied to escalation within a governed IT incident-resolution pipeline.

2.8 Escalation Behaviour in Automated Systems

A closely related work focuses on escalation decisions in conversational agents based on language models, analyzing when and why such systems defer to a human agent [15]. This is the only work in this domain that examines the question related to the central research problem of this dissertation. However, it does not analyze escalation decisions in the context of a multi-agent incident resolution system, and classifying failures using a taxonomy was also beyond the scope of this work.

2.9 Multi-Agent Failure Taxonomy (MAST)

Cemri et al. introduced the Multi-Agent System Failure Taxonomy, which was based on analyzing over 150 execution traces from 7 different multi-agent platforms, thus identifying 14 types of failures across three categories, namely system design flaws, inter-agent coordination failures, and task verification failures [12]. Further works then proceeded to apply the MAST taxonomy to classifying failures in multi-agent system execution employing an agreement-based approach [13] and to graph-guided multi-agent investigation [14], both of which demonstrated the taxonomy's versatility and ability to serve as a reliable assessment framework. However, there has been no application of MAST to examining a governed IT incident resolution pipeline, in order to assess whether introducing a dedicated governance agent could reduce the incidence of failure modes identified by the taxonomy.

2.10 Public Datasets and Benchmarking for IT Ticket Resolution

Researchers, who developed the Ticket-BERT propose that at the moment of research there was no big public labeled data set for the task of incident management, hence the necessity to use the enterprise's private ticket data [22]. However, recently some public databases became available for research including IT Service Ticket Classification Dataset [23] and Customer IT Support Ticket Dataset [24]. Moreover, nowadays AI assisted solutions already handle a considerable amount of tickets in the industry automatically, although mostly on simple and semi-complex level of incidents [25]. Such data sets allow the implementation of the governed multi-agent pipeline in practice without the use of enterprise's private data.

2.11 Multi-Agent Orchestration Framework Comparisons

Comparative research reports on multi-agent orchestration framework options, including LangGraph, CrewAI, AutoGen, and OpenAI Swarm, highlight that in terms of practical application needs such as auditability and rollback, the graph-based state model explicitly described by LangGraph better fits these requirements than similar solutions [26], [27]. Therefore, the LangGraph framework was selected as the orchestration layer.

2.12 Research Gap

Overall, the existing solutions for multi-agent incident-resolution systems and commercial platforms address the pipeline as described in the project overview, while calibration, conformal prediction, and escalation research separately address the question of confidence of LLMs in general. At the same time, the combination of these two topics, namely, applying a strict calibration-based escalation in a governed multi-agent incident-resolution pipeline and analyzing its impact through the lens of an already established multi-agent failure taxonomy, remains unexplored. Addressing this gap is the main aim of this project. We propose a system that bridges this gap by directly comparing the single-agent, ungoverned multi-agent, naively-governed multi-agent, and calibration-governed multi-agent pipelines on the same ticket data using MAST-based failure analysis to identify if and how calibrated governance improves system reliability.

3. Problem Statement

The recent rise of Artificial Intelligence (AI) and, specifically, Large Language Models (LLMs) has empowered Enterprise IT Service Management (ITSM) to implement solutions that address multi-faceted issues such as automating ticket categorization, diagnosis, and technical support. Recent advances in Multi-agent Large Language Model systems have demonstrated a certain degree of effectiveness, as such systems are designed to delegate specific tasks to different agents in order to scale up performance and bolster organizational decision-making. That being said, many implementations of such systems have proven to be ineffective at transitioning from proof-of-concept to a fully realized product.

A critical challenge of current multi-agent incident resolution pipelines is the inability to effectively govern autonomous decisions made by large language models (LLMs). In particular, most solutions adopt the LLM's own confidence score about its responses as the criterion for determining whether an incident should be resolved autonomously or escalated to engineers for further triage. However, recent studies have demonstrated that confidence scores of LLMs are often not calibrated well: many responses that an LLM is highly confident about are incorrect. Thus, governance strategies that only make decisions based on confidence scores often lead to unsafe resolutions, failure to escalate sufficiently complex incidents, and unwarranted escalation of simple incidents, which ultimately undermines user trust and the adoption of such systems in enterprise settings.

Another critical limitation involves the lack of specific mechanisms to evaluate failures in multi-agent frameworks. The Multi-Agent System Failure Taxonomy (MAST) was developed in 2025, which includes 14 types of failures inherent to autonomous multi-agent systems' functioning, ranging from communication breakdowns to verification and reasoning flaws. However, very few incident-management systems utilized such a useful tool to assess the effectiveness of their governance mechanisms in managing failures. As a result, it is difficult to estimate whether these systems' improvements could impact the failure modes described by the MAST Framework and effectively mitigate the occurrence of failures in AI-assisted incident management.

In addition, many existing enterprise AI systems provide little transparency or auditability. Enterprises demand full audit trails of all actions, decisions, confidence levels, documents pulled in for evidence, responses generated, validations passed or failed, and escalated items. Existing systems are optimized for automation, not auditability, and are therefore not fit for purpose in the enterprise.

Therefore, there is a need for developing a governance-aware, privacy-conscious, and auditable multi-agent architecture that achieves both automated resolution of IT incidents as well as calibrated confidence estimation, structured failure analysis, and reliable human-in-the-loop escalation, thus, improving the overall safety and trustworthiness of enterprise AI.

The proposed study will aim to answer the following research question:

"In a four-agent IT incident-resolution pipeline, do calibrated confidence based Governance Agent result in significantly lower MAST-taxonomy failure rates and improved safe human escalation compared to conventional baseline approaches that use confidence thresholds?"

4.Objectives

The primary objectives of the proposed project are as follows:

- Design and develop a four agent (Intake, Retrieval (RAG), Resolution, and Governance) AI pipeline, which will be used to automatically classify IT tickets, find relevant organizational information, generate solutions based on the retrieved information, and evaluate the correctness of the generated responses.
- Making a Governance Agent That Can Do Calibrated Confidence Estimation Instead of Using Naïve Raw Confidence Thresholds for Escalation Decisions, and Quantifying the Improvement Thereof with Standard Metrics Like Expected Calibration Error and Reliability Diagrams.
- The problem is to apply the Multi-Agent System Failure Taxonomy (MAST) as a means of structured evaluation by performing manual and semi-automatic labeling of execution traces to classify and quantify the occurrence of various failure modes during multi-agent execution.
- Build a deterministic and immutable audit trail encompassing all agent decisions, retrieved context, confidence scores, reasoning steps, and escalations, enabling explainability, traceability, human oversight, and responsible AI deployment.
- To evaluate the proposed system against standard baseline systems by using public IT support ticket datasets as well as adversarial synthetic tickets that can be leveraged to assess performance.
- To ensure key performance indicators (KPIs) of ticket classification accuracy, response quality, escalation accuracy, calibration, reduction of failures, and reliability.
- To design the system in such a way that it can be implemented as a modular solution and integrated into enterprise IT Service Management (ITSM) platforms and further expanded upon.
- To package the whole thing up as a reproducible, open source evaluation framework for the benchmarking of governance-aware multi-agent AI systems against enterprise incident management

5. Methodology

5.1 Proposed methodology

Phase 1: Literature review and requirement analysis
The research question is closed down against the literature reviewed, and the comparator set and evaluative criteria established in advance of undertaking the development work. This enables control over post-hoc modification of evaluation criteria, thus improving the consistency of the study.

Phase 2: Data acquisition and preprocessing
The datasets are cleaned, de-duplicated, and normalized. Each set is divided into training, calibration, and test sets in a 70:15:15 ratio. The synthetic set of troublesome tickets is created using LLM prompting for ambiguous, multi-issue, and insufficient tickets with escalation labels.

Phase 3: Agent and pipeline design
The pipeline is implemented as a LangGraph state graph, with Intake, Retrieval, Resolution and Governance agents operating as nodes in the graph with defined handoffs and fallbacks, prompts for each agent are tuned on a validation set, and the knowledge base for the Retrieval agent is indexed with either FAISS or Chroma.

Phase 4: Calibration module development
The calibration set is used to record the raw scores of the Resolution agent, which are then used to produce a single temperature value that best transforms the raw scores to an accurate confidence interval. The degree of calibration is evaluated using the Expected Calibration Error and reliability diagrams in comparison with the initial scores.

Phase 5: Escalation logic and audit trail
A ticket is escalated to a human when its calibrated confidence is below some chosen threshold, which is selected by testing different values on the calibration set, not chosen arbitrarily. All decisions of the agent, including confidence scores and subsequent escalation decisions, are logged in a timestamped audit trail for reference.

Phase 6: MAST failure labeling
Execution traces of all pipeline variants are collected, and for a subset of between 100-150 traces a manual labeling is carried out by two annotators using the failure categories of the MAST. An LLM based annotator is then validated on this subset and applied to the remaining traces.

Phase 7: Comparative evaluation
All four options are tested on the same set of data including the artificial set and the resolution quality, escalation accuracy, calibration error, and failure frequency are evaluated. Several trials are carried out and statistical tests are made to confirm that the difference is significant.

Phase 8: Deployment and demonstration
The final pipeline is then containerized with Docker and deployed to a local Kubernetes cluster to demonstrate the solution in action, while a FastAPI dashboard is prepared to illustrate the ticket flow, confidence levels and escalation history so that it is more engaging than a simple

presentation of slides.

5.2 System Architecture

The proposed system will implement a multi-agent architecture that would allow for fully automated operation of a business process that is currently managed manually. By breaking down the end-to-end resolution process into component stages handled by specialized agents, the system would be easier to scale, maintain, troubleshoot, and audit than if the entire process was automated by a single LLM. The system will use a specialized agent for each step of the ticket resolution lifecycle, thus creating a reliable audit trail inside the system.

The system can be broken down into the following five components:

A. **Intake Agent**

B. **Retrieval Agent**

C. **Resolution Agent**

D. **Governance Agent**

E. **Evaluation Harness**

A. Intake Agent
The Intake Agent is the first stage of the proposed system, which is primarily responsible for receiving incoming IT support tickets and transforming raw natural language data into a suitable format for the majority of the following agents. The agent undertakes several Natural Language Processing tasks when handling each support ticket, including extracting information from the body of the message, identifying the category and priority of the ticket, extracting operating systems, applications, error codes, network-related issues, and devices mentioned in the ticket, and determining keywords that will help with later retrieval. The processed output, which normally contains ticket category, priority, keywords, confidence score, and additional structured data, is then passed to the Retrieval Agent.

B. Retrieval Agent
After classification, the Retrieval Agent is used to find the technical knowledge needed to resolve the reported issue, typically using a Retrieval-Augmented Generation approach, instead of relying solely on the language model's own knowledge. The ticket embedding is compared to a vector database containing public IT support documentation, troubleshooting guides, FAQs, technical knowledge base articles, resolved tickets, and procedures, and the related context is retrieved and ranked by relevance, for subsequent use by the Resolution Agent. The Resolution Agent grounds responses by reducing hallucination risk and increasing factual response accuracy by using the context retrieved by the Retrieval Agent, focusing on semantic document search, context ranking, evidence collection, and preparation for reasoning.

C. Resolution Agent
The Resolution Agent produces a possible solution based on the initial request of the user alongside the documents the Retrieval Agent has found relevant to the issue. With the produced response, it calculates a similarity score to the actual response of the user, found in the provided data set. It then uses an LLM-as-a-judge method to evaluate the solutions accuracy, relevance, quality, and completeness. The solution, along with its reasoning, documentation, approximate confidence score, and summary is then sent to the Governance Agent for validation before being delivered as the final response.

D. Governance Agent
The Governance Agent is the core innovation of the proposed solution. Rather than simply answering queries, it evaluates if the provided solution is confidently enough correct that it can be safely returned or if the query should be escalated to an engineer for refinement, using confidence calibration approaches such as temperature scaling, split-conformal prediction, confidence normalization, and risk-

aware threshold selection on a held-out validation set to make this determination. It also checks for adherence to policy, completeness and correctness of the response, retrieval of relevant supporting evidence, satisfaction of safety requirements, and need for escalation while maintaining an auditable trail that stores ticket ids, timestamps, agent decisions, and retrieved documents for every run to enable enhanced explainability, accountability, debugging, and regulatory compliance.

E. Evaluation Harness

The Evaluation Harness conducts tests and benchmarks the whole system after the execution instead of during the ticket processing by analyzing the traces produced from execution to assess the performance of every pipeline configuration. It handles replaying of the execution traces, comparing of the pipeline configurations, calibration performance evaluation, detection of failure patterns, and generating reports, from which the quantitative measures necessary for the comparison of the proposed solution with the baseline are extracted.

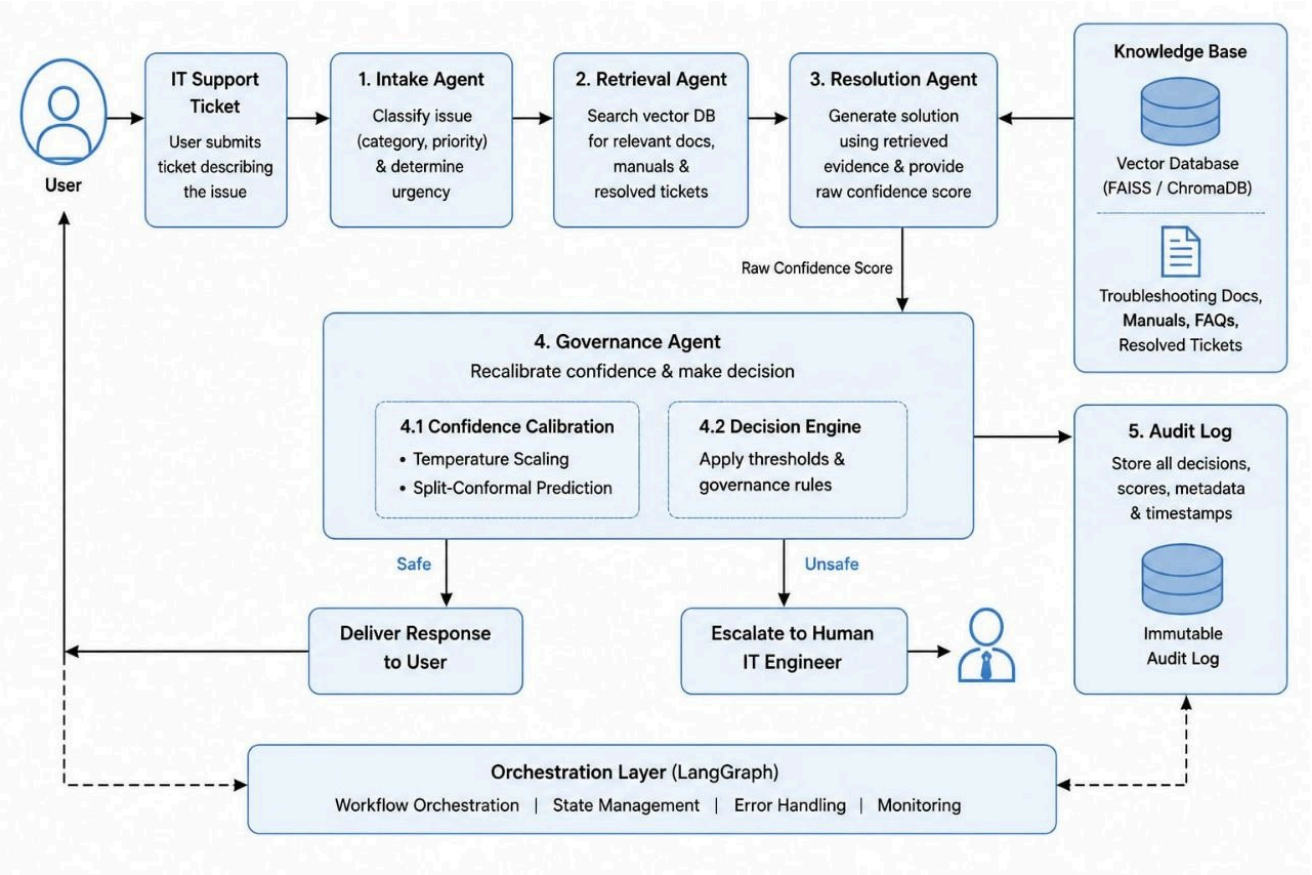


Figure 5.1 : System Architecture

5.3 Evaluation Metrics

The proposed system will be evaluated using multiple quantitative and qualitative performance metrics.

A. Resolution Quality

Resolution quality measures how closely the generated solution matches the actual human-written solution contained in the dataset.

B. Escalation Quality

The Governance Agent must correctly determine whether a ticket should be solved automatically or escalated.

C. Confidence Calibration

The effectiveness of confidence calibration will be measured using:

Lower calibration error indicates better confidence estimation.

D. Failure-Mode Frequency

Using the MAST taxonomy, every execution trace will be analyzed to determine which failure categories occurred.

The frequency of each failure mode will be calculated separately for all four pipeline variants.

This evaluation demonstrates whether the Governance Agent actually reduces known multi-agent failure patterns.

E. Computational Cost and Latency

Enterprise AI systems must balance accuracy with computational efficiency.

Therefore, the following operational metrics will also be recorded:

- Total input tokens
- Total output tokens
- Overall API cost
- Average execution time

6. Hardware/Software Requirement

Hardware Requirements:

1. Processor: Intel Core i5 (10th generation or higher) or AMD Ryzen 5 equivalent, or better

2. RAM: 8 GB minimum, 16 GB recommended

3. Storage: 20 GB free disk space for datasets, models, logs, and containers

4. GPU: Not mandatory as LLMs are accessed via API; an entry-level GPU is optional only for running a local fallback model

5. Internet: Stable broadband connection for API calls, dataset downloads, and experiment tracking

6. Display: Standard monitor for development and dashboard testing

Software Requirements:

1. Operating System: Windows 10/11, Ubuntu 20.04 or later, or macOS

2. Programming Language: Python 3.10 or higher

3. Orchestration Framework: LangGraph for building the multi-agent state graph

4. LLM Access: Gemini API as the primary language model, with a local open-weight model via Ollama as the deterministic fallback

5. Retrieval and Vector Store: FAISS or Chroma for the knowledge base

6. Backend Framework: FastAPI to design the ticket intake endpoint and dashboard API

7. Experiment Tracking: MLflow to organize pipeline runs, calibration parameters, and metrics

8. Machine Learning Libraries: scikit-learn, pandas, numpy, and matplotlib

9. Database: PostgreSQL or SQLite for the audit log

10. Containerization and deployment: Docker, and Kubernetes via Minikube or Kind

11. Development Tools: Visual Studio Code or PyCharm

12. Version Control: Git and GitHub

13. Dataset Sources: Kaggle API for the IT Service Ticket Classification Dataset and the Customer IT Support Ticket Dataset

7. Project Schedule :

Calendar of Project Planning:

Duration	Phase	Key Activities
Week 1-2	Literature Review & Requirement Analysis	Study existing systems, finalize the research question, freeze the comparator set and evaluation metrics before development starts
Week 3-4	Data Acquisition & Preprocessing	Collect and clean datasets, split into training, calibration and test sets, generate the synthetic adversarial ticket set
Week 5-7	Agent & Pipeline Design	Build the LangGraph pipeline with Intake, Retrieval, Resolution and Governance agents, design and refine prompts, set up the knowledge base
Week 8-9	Calibration Module Development	Record baseline confidence scores, apply temperature scaling, evaluate using Expected Calibration Error and reliability diagrams
Week 10-12	Escalation Logic, Audit Trail & MAST Labeling	Set the escalation threshold, build the audit log, label execution traces against the MAST failure categories
Week 13-14	Comparative Evaluation	Test all four pipeline variants on the full dataset, compute all metrics, run statistical tests to confirm results
Week 15-16	Deployment & Documentation	Containerize with Docker, deploy on Kubernetes, build the dashboard, finalize the project report and

8. References :

1. Microsoft Research. 2025. "Triangle: Empowering Incident Triage with Multi-LLM-Agents." In Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2025).

2. "Multi-Agent LLM Orchestration Achieves Deterministic, High-Quality Decision Support for Incident Response." 2026. arXiv preprint arXiv:2511.15755. https://arxiv.org/abs/2511.15755

3. "Intelligent Site Reliability Engineering: A Multi-Agent LLM Framework for Automated Incident Analysis and Root Cause Determination." 2025. ResearchGate.

4. "LLM Agents Security Duality: A Comprehensive Survey of Self-Security and Empowered Cybersecurity." 2026. arXiv preprint arXiv:2606.28450. https://arxiv.org/abs/2606.28450

5. PagerDuty Inc. 2026. "PagerDuty Operations Cloud Spring 2026 Release Notes." PagerDuty Newsroom.

6. ServiceNow Inc. 2026. "ServiceNow Completes Acquisition of Moveworks; Launches EmployeeWorks." ServiceNow Newsroom.

7. systempromptio. 2026. "awesome-ai-agent-governance." GitHub repository.

8. Strata Identity. 2026. "Human-in-the-Loop: A 2026 Guide to AI Oversight." strata.io

9. Digital Applied. 2026. "Human-in-the-Loop Escalation Design for AI Agents 2026."

10. Zylos AI Research. 2026. "AI Agent Governance and Compliance in 2026." zylos.ai

11. Zylos AI Research. 2026. "LLM Calibration and Uncertainty Quantification in Production AI Agents." Zylos.ai.

12. Cemri, M., et al. 2025. "Why Do Multi-Agent LLM Systems Fail?" arXiv preprint arXiv:2503.13657. https://arxiv.org/abs/2503.13657

13. "The Six Sigma Agent: Achieving Enterprise-Grade Reliability in LLM Systems Through Consensus-Driven Decomposed Execution." 2026. arXiv preprint arXiv:2601.22290. https://arxiv.org/abs/2601.22290

14. "Think Locally, Explain Globally: Graph-Guided LLM Investigations via Local Reasoning and Belief Propagation." 2026. arXiv preprint arXiv:2601.17915. https://arxiv.org/abs/2601.17915

15. "Act or Escalate? Evaluating Escalation Behavior in Automation with Language Models." 2026. arXiv preprint arXiv:2604.08588. https://arxiv.org/abs/2604.08588

16. "Calibrating Conservatism for Scalable Oversight." 2026. arXiv preprint arXiv:2605.28807. https://arxiv.org/abs/2605.28807

17. "Conformal Constrained Policy Optimization for Cost-Effective LLM Agents." 2026. arXiv preprint arXiv:2511.11828. https://arxiv.org/abs/2511.11828

18. "Conformal Agent Error Attribution." 2026. arXiv preprint arXiv:2605.06788. https://arxiv.org/abs/2605.06788

19. Guo, C., G. Pleiss, Y. Sun, and K. Q. Weinberger. 2017. "On Calibration of Modern Neural Networks." In Proceedings of the International Conference on Machine Learning (ICML).

20. Vovk, V., A. Gammerman, and G. Shafer. 2005. Algorithmic Learning in a Random World. New York: Springer.

21. Peffers, K., T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. 2007. "A Design Science Research Methodology for Information Systems Research." Journal of Management Information Systems 24 (3): 45-77.

22. "Ticket-BERT: Labeling Incident Management Tickets with Language Models." 2023. arXiv preprint arXiv:2307.00108. https://arxiv.org/abs/2307.00108

23. Goh, A. 2024. "IT Service Ticket Classification Dataset." Kaggle.

24. Bueck, T. 2025. "Customer IT Support - Ticket Dataset." Kaggle.

25. Unthread.io. 2026. "23 Support Ticket Resolution Statistics by Complexity (2026)."

26. DEV Community. 2026. "LangGraph vs CrewAI vs AutoGen: The Complete Multi-Agent AI Orchestration Guide for 2026."

27. Presenc AI Research. 2026. "Multi-Agent Orchestration Frameworks 2026 (LangGraph, CrewAI, AutoGen, Swarm)."